UNITÀ 2 · SVILUPPO DI UN WEB SERVICE RESTFUL IN JAVA

BiblioAPI: arrivano i prestiti

La biblioteca è contenta dei libri, ma ora vuole sapere chi li ha in mano. Estendi il BiblioServer scritto in classe con una nuova risorsa: i prestiti.

Cosa consegni: un file **biblioapi-prestiti-cognome.zip** con **BiblioServer.java** e **prove.txt** su Novaula, nella scheda Compiti. **Scadenza:** quella indicata su Novaula.

Le nuove richieste

Richiesta	Che cosa fa	Risposte
`POST /api/libri/{id}/prestiti` con `{"studente":"Rossi Anna 5G"}`	Presta il libro	201 + Location · 400 · 404 libro inesistente · **409** se è già in prestito
`GET /api/prestiti`	Elenca tutti i prestiti	200
`GET /api/prestiti?attivi=true`	Solo quelli non restituiti	200
`PUT /api/prestiti/{id}` con `{"restituito":true}`	Registra la restituzione	200 · 404
`GET /api/libri?autore=Calvino`	Filtra i libri per autore	200, anche con `[]`

Che cosa devi fare

1. Crea il record `Prestito(int id, int libroId, String studente, String data, boolean restituito)` con il suo `toJson()`. Per la data usa `LocalDate.now().toString()`.
2. Usa una `ConcurrentHashMap` e un `AtomicInteger` separati da quelli dei libri. Registra il contesto `/api/prestiti`.
3. Gestisci le richieste della tabella con i codici indicati. Un libro con un prestito attivo non si può prestare di nuovo; dopo la restituzione sì.
4. Bonus (1 punto): `DELETE /api/libri/{id}` di un libro in prestito risponde 409.
5. In **prove.txt** incolla l'output di `curl -i` di almeno 8 richieste, comprese un 409 e un 404, nell'ordine in cui le hai fatte.

Esempio

`POST /api/libri/2/prestiti` → `201`, `Location: /api/prestiti/1`, corpo `{"id":1,"libroId":2,"studente":"Rossi Anna 5G","data":"2026-11-05","restituito":false}`. Lo stesso POST subito dopo → `409` con `{"errore":"libro già in prestito"}`.

Curiosità. Il codice 409 Conflict è definito nella RFC 9110: si usa quando la richiesta è corretta ma è in conflitto con lo stato attuale della risorsa. È il codice tipico di prenotazioni doppie, nomi utente già presi e modifiche concorrenti.

Come viene valutato

Criterio	Punti
Il server compila e si avvia con `java BiblioServer.java`	1
Creazione del prestito con 201, 400, 404 e 409 corretti	3
Elenco, filtro `attivi=true` e restituzione con PUT	2
Filtro dei libri per autore	1
JSON valido, strutture thread-safe, codice ordinato e commentato	1
prove.txt completo e coerente	2

Il bonus vale 1 punto oltre il 10. Se usi un assistente AI, scrivilo in un commento in cima al file e indica che cosa hai corretto tu.